

CSD380: DevOps

Module 2: DevOps

Assignment 2: Case Study: Operation InVersion at LinkedIn (2011)

Isaac Ellingson

6/14/2026

How do I summarize something already so short?

LinkedIn started in 2003. They had a clear plan, hit a gap in the market, and less than one year later they passed a million members. And then they just kept racking up millions of users. By 2011, they had built systems that could cope with this scaling, but they had to really rush to keep up with user needs.

The system they initially built was called Leo. It was a single Java application serving pages and talking to the database. The fix they made to scale up to a million users and beyond was to do user graph and search entirely in-memory. This worked in two interlocking pieces outside of Leo. This worked... kind of. Leo itself kept crashing.

Since just adding CPUs and memory wasn't meeting their needs, whenever they wanted to add a feature, they once again wrote it as a kind of microservice outside of Leo. This cluster operated efficiently, allowed them to move services to different hardware, and by 2010 they had over a hundred such services floating around. But Leo kept crashing, it took forever to release new versions of it, and everyone was frustrated by bad deployments and dev time that *should have been spent innovating*, lost to the dev hole of dealing with Leo's quirks.

But you can't just kill Leo. That's the core of your business logic, it's a big investment to replace. Kevin Scott, new to the company but not new to technical debt, declared that there would be no new features for two months. They would revamp the software, and also the software process that got them into this trouble.

This started by creating and collecting tools to help them develop code. Automated testing, test deployment, and production deployment meant updates could reach the main codebase faster and with fewer problems. And then taking the dev time to deal with the software part of the technical debt let them focus on new features without it crashing, slowing down, or getting in the way. Once they were done, they were releasing three major updates a day, while doing fewer late-night crunches. Developers were happier, and the software got better.

The point of the aside, if there is one, is that this was a risky, last-ditch effort. It worked, but it only kept working because they changed their process and not just their software. Everyone walks on this edge between moving fast to catch up with the market, and making wise technical decisions. It's much easier to manage this problem, little by little, cleaning as you go, rather than stopping the world for two months so your team doesn't crash and burn.

References

While the assignment was to summarize the book, I attempted to track down the book references, and in doing so found some other elegant material that deepens my understanding of the case study as explained in the book.

Kim, G. (2023, November 12). Case study: LinkedIn's 2011 operation InVersion through lens of slowify, simplify, and amplify. Retrieved from LinkedIn.com website:
<https://www.linkedin.com/pulse/case-study-linkedins-2011-operation-inversion-through-gene-kim-dht2c>

Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). *The DevOps handbook, second edition*. IT Revolution.

Scott, K. (2017, July 20). When your tech debt comes due. Retrieved from LinkedIn.com website: <https://www.linkedin.com/pulse/when-your-tech-debt-comes-due-kevin-scott>

Vance, A. (2013, April 12). Inside operation InVersion, the code freeze that saved LinkedIn. Retrieved from Bloomberg.com website:
<https://www.bloomberg.com/news/articles/2013-04-10/inside-operation-inversion-the-code-freeze-that-saved-linkedin>